JARVIS
MASTER HANDOVER — VERSION 4

Continuity document for VS Code coding agents
Free models & packages only • Human approval gates • PC hardware limits

Prepared 30 October 2026 23:06 UTC
Supersedes Version 3 for agent startup and next-action authority

Purpose	A new VS Code chat/agent that is given only this document must be able to continue the project safely and stop o
Authority	This V4 is the authoritative current-state, constraint, and next-action source. Detailed history remains in V3 / proje
Model constraint	FREE models and FREE/open-source packages only.
Hardware	Must run on the recorded PC specifications (Section 5).
Microphone	Exists but is NOT currently connected. No voice work.
Human role	Final authority. Limited coding knowledge — agents must be explicit and safe.

START HERE — NEW VS CODE AGENT

If you are a new VS Code chat or coding agent and have been given this document, follow the instructions in the box below exactly. Do not skip steps. Do not begin implementation until the human answers the direction question.

MANDATORY STARTUP INSTRUCTIONS

```
Read this entire document (JARVIS MASTER HANDOVER VERSION 4) completely first.
It is the authoritative continuity and constraint document.

Then follow its Section 3 Startup sequence exactly:
1. Confirm you have read Version 4.
2. Check live git status, branch and HEAD (no destructive commands).
3. Note any discrepancy with the snapshot in this document.
4. Ask the human the mandatory direction question (see below) and WAIT for the answer.

Do not implement anything, change any files, run tests, install packages, or expand scope
until the human gives a clear scoped instruction.

Hard constraints you must obey:
- Free models and free/open-source packages only.
- Design within the recorded PC specifications (i7-8700K, 32 GB RAM, GTX 1080 Ti 11 GB, Windows).
- Microphone exists but is NOT connected — no voice work.
- Stop and ask whenever anything is ambiguous or outside the approved slice.

After the human answers the direction question, proceed only within the scope they approve
and report using the structure in Section 7 of this document.
```

Mandatory direction question (ask exactly this, then wait):

```
Would you like me to continue from the last recommended milestone
(Phase 5.3.1 Read-only Testing Awareness), or would you like to
change direction? Please state any scope limits or new constraints.
```

Until you receive a clear scoped answer, your only permitted actions are: read this document, inspect git status non-destructively, and ask the question above.

1. Hard Constraints (Non-negotiable)

1.1 Free models and packages only

- Only free local models (Ollama) and free/open-source packages are permitted.
- Current primary model path: local Ollama (llama3.2 or whatever is installed and working).
- Do not introduce paid APIs, commercial SDKs, or services that require payment or API keys the human has not supplied.
- If a capability appears to need a paid service, stop and report the limitation. Propose free alternatives if any exist.

1.2 Hardware limits (must design within these)

Component	Recorded value	Implication
CPU	Intel Core i7-8700K @ 3.70 GHz	Capable but not newest; prefer efficient models
RAM	32 GB DDR4	Comfortable for 7B–13B local models
GPU	NVIDIA GTX 1080 Ti 11 GB	Useful for local inference; respect VRAM
Storage	≈1 TB SSD (sources conflict on exact size)	Keep models and logs under control
OS	Windows 64-bit	Primary target platform
Python	3.x (documentary claim 3.15)	Use existing environment; no forced major upgrades

All design and package choices must remain practical on this machine.

1.3 Microphone status

A microphone is present but is **not currently connected**. Voice input, wake-word, STT, TTS and related work are out of scope until the human connects the microphone and explicitly authorises voice work.

1.4 Safety and autonomy limits

- No autonomous code modification, commit, push, merge, or deployment.
- No execution of tests, shell commands, or package installs without explicit scoped approval for that action.
- No expansion into voice, smart-home, multi-agent orchestration, or durable cloud routing unless newly authorised.
- When in doubt, stop and ask. Prefer stopping early over risking unintended changes.

2. Project Identity and Current State

2.1 Identity

Term	Definition
Project	Jarvis — the personal desktop AI assistant and overall project.
Repository	AI-Agent — contains the Jarvis runtime, Developer Agent subsystem, tests, and docs.
Developer Agent	A supervised subsystem for repository/architecture awareness and human-reviewed proposals. Not the whole project.
This document	Version 4 continuity layer. Authoritative for current state, constraints, and next authorised work.

2.2 Snapshot

Field	Value
Latest implemented capability	Phase 5.2.4 Proposal Review Workflow
Latest recorded full test run	127 tests passed (2026-10-30); not automatically re-run by this document
Repository baseline (from V3)	main @ 31b4dd8d059bcc56733622b20aa7f3f9b94f999d
Current development focus	Awaiting human decision on next slice (see Section 4)
Voice / wake word / STT / TTS	Not implemented; microphone not connected
Durable long-term memory	Not implemented
Smart-home / Home Assistant	Not implemented
General vision / OCR	Screenshot capture only; interpretation not established
Multi-agent orchestration	Not implemented
Hybrid / cloud model routing	Local Ollama only; cloud requires explicit future approval

Detailed history remains in Version 3 and project Markdown. This V4 overrides only on constraints, next-action authority, and agent behaviour.

3. Startup Sequence (Detailed)

This is the expanded form of the mandatory instructions on page 2. Follow every step.

- Read this entire Version 4 document.
- If more historical detail is needed, consult Version 3 and the project Markdown files (reading order in Section 8).
- Check live repository state: branch, status, HEAD. Use only non-destructive git commands.
- Compare live state to the snapshot in Section 2.2 and note any discrepancies in your first reply.
- Ask the mandatory direction question (exact wording on page 2) and wait for a clear human answer.
- Only after scoped approval, perform the smallest safe step inside that scope, then report using Section 7.

4. Next Authorised Work

This is the single most important scope section. Do not invent additional work.

4.1 Default next slice (recommended — not yet approved for implementation)

Phase 5.3.1 — Read-only Testing Awareness

Inventory test files, frameworks/configuration, fixtures and platform constraints from an explicitly selected repository snapshot. For a supplied proposal, map relevant unit/integration/regression/performance/manual checks, expected outcomes, side effects, uncertainty and approval needs.

Strict limits of this slice:

- Read-only. Do not execute any tests, commands, subprocesses or test frameworks.
- Do not invent runnable commands if configuration is missing or malformed — report incompleteness instead.
- Separate states clearly: test present / proposed / approved / blocked / not-run / actually executed.
- Add focused tests only for the inventory and mapping behaviour itself, and only after explicit implementation approval.

The human may change direction. Whatever they authorise becomes the only permitted scope until further notice.

5. PC Specifications (Design Limit)

Every design and package choice must remain practical on this machine. Do not assume newer hardware.

Component	Value
CPU	Intel Core i7-8700K, 3.70 GHz
RAM	32 GB DDR4
GPU	NVIDIA GeForce GTX 1080 Ti, 11 GB VRAM
Storage	Approximately 1 TB SSD (exact size sources conflict)
OS	Windows 64-bit
Primary AI runtime	Ollama, local (llama3.2 or currently installed model)

6. Stop Conditions and Approval Gates

Stop and request explicit human approval before any of the following:

Trigger	Required action
Any file modification outside an already-approved narrow slice	Stop. Describe exact files and intent. Wait.
Running tests, shell commands, or installing packages	Stop. Show the exact command(s). Wait.
Changing architecture, adding tools, or expanding scope	Stop. Explain and propose the smallest safe alternative.
Touching voice, microphone, smart-home, multi-agent, or cloud routing	Stop. Out of scope until newly authorised.
Git commit, push, merge, or branch operations	Stop. Separate explicit approval required for each.
Uncertainty about scope or safety	Stop. Prefer early stop over risk.
Any paid service or non-free package appears necessary	Stop. Report limitation and free alternatives if any.
Assumptions that exceed recorded PC specs	Stop. Redesign for available machine or ask.

Design discussion ≠ implementation approval. Implementation approval is scoped and does not include commit or push. The human has limited coding knowledge — be plain and concrete.

7. Required Reporting Format

After any approved work (or when stopping), use this structure:

```
FILES CHANGED:
- [list or “none”]

WHAT WAS DONE:
- [concrete summary]

WHAT WAS NOT DONE / LEFT FOR LATER:
- [list]

TESTS PERFORMED:
- [exact commands and results, or “none – not approved”]

TESTS NOT PERFORMED:
- [list]

RISKS / OPEN QUESTIONS:
- [list]

NEXT RECOMMENDED STEP (not started):
- [one clear recommendation]

WAITING FOR YOUR APPROVAL ON: [specific item or “none”]
```

8. Reading Order

- This document — Version 4 (constraints, state, next action, stop rules, startup instructions).
- CURRENT_PROJECT_STATE.md (if present).
- PROJECT_CONTINUITY_LOG.md or NEW_AI_STARTUP_CHECKLIST.md (if present).
- PHASE5_REVIEW.md and PHASE5_3_OPTIONS.md (for the recommended next slice).
- PROJECT_VISION.md and ROADMAP.md (context only; older status text may be stale).
- Relevant phase design / implementation plan and the specific source + test files for the approved slice.
- Version 3 Master Handover for deep archive when needed.

9. Implemented vs Not Implemented (Quick Reference)

9.1 Implemented (evidence-backed)

- Structured JSON actions, validation, sequential task execution, verification states, target/window awareness.
- Narrow desktop tools: open Notepad, screenshot, mouse move, list windows, type text.
- Tkinter GUI request path using local Ollama.
- Repository awareness and architecture awareness (bounded, read-only, in-memory).
- Proposal representation, construction, validation, and review workflow (human disposition required).
- 127-test suite recorded as passing on 2026-10-30 (not re-run by this document).

9.2 Explicitly not implemented

- Voice interface, wake word, STT, TTS (microphone not connected).
- Durable long-term / semantic / knowledge-graph memory.
- Smart-home or Home Assistant integration.
- General vision, OCR, or semantic screen understanding (capture only).
- Automated test execution service, sandbox validation, Capability Evolution Engine.
- Multi-agent orchestration, application generation, autonomous code modification.
- Durable proposal store / review UI, hybrid cloud model routing.
- Mobile companion, multi-room satellites, proactive home monitoring.

10. Long-term Vision: Jarvis Driving Development (Future Only)

ASPIRATIONAL — not current work. Do not implement any of this until the human gives a new, explicit authorisation.

The human's longer-term goal is that Jarvis becomes capable enough to understand project state, propose the next safe slice, drive a VS Code coding agent under strict rules, request high-level approval from the human, and (after that high-level approval) give lower-level approval to the coding agent for the already-scoped work. The human remains final authority. This capability is not to be built until foundations, memory, audit and safety gates are mature.

11. Document Control

Title: JARVIS MASTER HANDOVER — VERSION 4
Created: 2026-10-30 23:06 UTC
Replaces: Version 3 as the authoritative agent-startup and next-action document.
Retention: Keep Version 3 as historical archive.
Update rule: When the human authorises a new slice or changes constraints, a new continuity snapshot should be written.

How a human starts a new VS Code agent session: give the agent only this PDF (or point it at this file). The agent must follow the START HERE instructions on page 2. No extra paste text is required from the human.

End of Version 4. A new VS Code agent that has been given this document must now execute the START HERE block on page 2.